AutoSQUID Operating Protocol

Ran Lab — SQUID magnetometry

June 2026

Abstract

This document describes how to acquire SQUID noise traces with the `AutoSQUID` package, and serves as a reference for the instrument configuration. The logic lives in a small Python package (`AutoSQUID/`); two thin notebooks drive it — a measurement-cycle notebook for acquisition and `auto_s_tune.ipynb` for centering the locked output before a run. The committed showcase is `example_measurement_cycle.ipynb` (a generic template); your lab's `measurement_cycle.ipynb` (with your `temp_reader` and data paths) is kept local / gitignored. All settings are held in a single `Config` object built in the notebook; every library function takes that `cfg`. For one fixed temperature, and for each scan interval requested, the cycle collects a target number of clean, gap-free time-series runs at the chosen sample rate; it watches for a flux-lock jump while acquiring, resets the flux-locked loop and re-acquires on a failure, logs the mixing-chamber temperature throughout, and saves each trace in the standard `PCS102` text format with an append-only run ledger. Section 1 is the configuration; Sections 2–3 the two instruments; Section 4 the per-interval run loop; Section 5 the acquisition procedure; and Section 6 the auto-S-tune centering procedure.

Contents

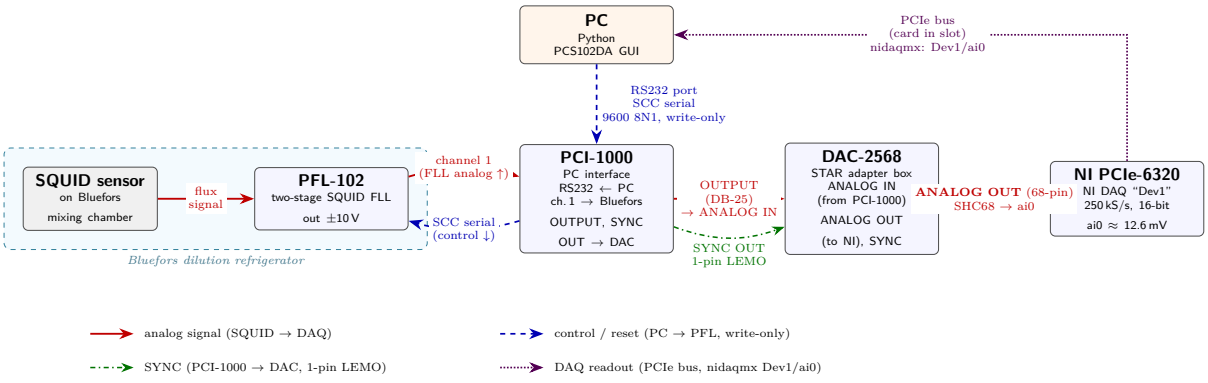


Figure 1: Wiring of the SQUID acquisition rig. Analog: SQUID/PFL-102 (Bluefors) → PCI-1000 ch.1 → PCI-1000 OUTPUT → DAC-2568 ANALOG IN → ANALOG OUT (68-pin) → PCIe-6320 ai0, read over the PCIe bus via nidaqmx. Control: PC → PCI-1000 RS232 → PFL (SCC serial, write-only). Sync: PCI-1000 SYNC OUT → DAC-2568 (1-pin LEMO).

1 Configuration parameters

All run settings are fields of one `Config` object, built in the notebook's first cell (`cfg = sq.Config(...)`); every library function takes that `cfg`. The table below explains each field; the defaults shown are the shipped values. A few derived values are computed properties of `cfg` (the scan-interval list, the file-name tag, the output directory, and the PFL register), so there is no duplicated state to keep in sync.

`cfg` field	Meaning	Default

cfg field	Meaning	Default
Acquisition		
scan_interval_s	Sample interval(s) in seconds; one full cycle runs per entry (a single number or a list). Sets the sample rate $f_s = 1/\text{interval}$ (4/20/100/500 μs $\rightarrow$ 250/50/10/2 kHz).	[100e-6]
n_points	Number of samples in each consecutive run.	10_000_000
temp_label	"auto" reads, validates, and rounds the current MXC temperature into the file name (a missing / non-finite / non-physical reading aborts before acquiring); or a literal such as "14mK".	"auto"
n_trials	Target number of *clean* traces to collect per scan interval.	2
Live jump check (during acquisition)		
chunk	Points per streamed read; this is the jump-check cadence. It does *not* break the gap-free run (see Section 2).	100_000
jump_v	Baseline-mean slip threshold (V): below the ± 1 V clip, above clean drift.	0.5
rail_v	True-rail catch, $\lvert V \rvert >$ this. Kept at 9.5 (range-agnostic); at ± 1 V the ADC clips below it, so a clipped slip reads as a jump.	9.5
baseline_chunks	Opening chunks that set the baseline mean μ_0 (the first chunk).	1
Temperature logging		
temp_every_s	Background temperature sampling period, in seconds.	30
temp_channel	Thermometer channel passed to temp_reader; 6 = mixing chamber (MXC).	6
temp_reader	*Lab-specific*, injected in the notebook: fn(channel) -> T in K (wrap your thermometer). None until set; required for any run with temp_label="auto".	None
Serial control (SCC)		
port	Serial COM port for control. PCS102DA can stay open on COM2 / another channel to hold S-lock.	"COM3"
channel	SCC address = the locked PFL channel.	1
reset_opcode	SCC opcode for the feedback-loop register.	0x50
baud	Serial speed (8N1: 8 data bits, no parity, 1 stop bit).	9600
register_override	Force a specific PFL register; None uses the standard SQUID-locked 0x408.	None
Verify / DAQ / output		
verify_n, verify_fs	Samples and rate of the post-reset verify read.	1000, 10_000
scan_n, scan_fs	Samples and rate of the channel-liveliness probe.	4000, 50_000
baseline_v	|mean| below this counts as a cleared / locked baseline.	0.10
vrange	DAQ analog-input range, in volts (Section 2).	1.0
max_attempts	Maximum total acquisitions per interval (clean + failed).	4
force_device, force_ai	Override the device / channel auto-pick.	None
daq_ai	NI analog-input carrying the output; empty until the channel auto-detect (detect_ai_channel) sets it, or set force_ai.	""
data_root	*Lab-specific* data root; outdir = data_root / user / date. Set per install.	""

cfg field	Meaning	Default
`user`	Operator name; selects the data folder.	`""`
`date`	Date subfolder (`""` = the bare `user` folder).	`""`

Estimated time per interval ≈ `n_points` × interval (≈1000 s at 100 μs, ≈5000 s at 500 μs). One notebook run covers one temperature setpoint and all the scan intervals listed; a new temperature is a new run.

2 The PCIe-6320 DAQ card

A National Instruments PCIe-6320 (16-bit multifunction DAQ, up to 250 kS/s) digitizes the analog flux-locked-loop output. The signal reaches the card on `ai0` via the PCI-1000 output and the DAC-2568 adapter. Because the maximum rate is 250 kS/s, the 4 μs scan interval (250 kHz) sits exactly at the card's ceiling.

2.1 Input range (`vrange`) and why it sets precision

`vrange` is the *card's* programmable analog-input range, not a PFL-102 setting. It is set to ±1 V — the range the NI card was set to for the earlier measurements on this rig. The 6320 offers four discrete ranges — ±0.2, ±1, ±5, and ±10 V — and the driver rounds the requested interval up to the smallest range that contains it.

The converter is 16-bit, so it always produces $2^{16} = 65{,}536$ codes *regardless* of range; the range is simply the voltage span those fixed codes are stretched across:

$$\text{LSB} \;=\; \frac{2 \times \texttt{vrange}}{65{,}536}.$$

A smaller range therefore gives finer steps (±10 V →∼305 μV/code; ±1 V →∼30.5 μV/code) at the cost of headroom. It is a strict dynamic-range vs. resolution trade at fixed bit depth.

Caution. This notebook uses ±1 V (the range the NI card was set to for the earlier measurements). The clean signal is ∼12 mV mean with ∼2 mV noise, so ±1 V gives ∼30 μV/code — ∼10× finer than ±10 V — with ample headroom over the signal. The trade is that the ADC *clips* at ±1 V, so the thresholds account for it: `jump_v` is lowered to 0.1 V to catch an in-range flux-slip by its mean deviation, while `rail_v` stays at 9.5 V — at ±1 V the ADC clips at ∼1 V (below 9.5), so the rail check never fires and a clipped slip is reported as a jump; the same notebook then still flags a true rail if the card returns to ±10 V.

2.2 Terminal configuration: RSE

The channel is read in **RSE (Referenced Single-Ended)** mode: each input is measured relative to the board's analog ground (AIGND), one wire per channel. This matches the single-ended cable from the DAC-2568 analog output to `ai0`. The alternatives are *NRSE* (measured against a shared floating sense line) and *DIFF* (measured against a paired negative input, which rejects common-mode / ground-loop noise but needs a differential pair and halves the channel count). DIFF would improve noise rejection, but the existing cable is single-ended, so RSE is the correct match.

2.3 Sampling: one gap-free run, drained in chunks

Each acquisition is a *finite* task with `samps_per_chan = N_POINTS`: one hardware-timed sample clock fills a single buffer for the whole run. That buffer is then drained in `chunk`-point reads so the live jump check can run every `chunk` points. Chunking is only a read pattern — adjacent chunks are one sample period apart — so the run remains a single, consecutive, gap-free acquisition. Because the buffer holds the entire run, a slow read cannot cause an overflow. When a live check fails the task is stopped early (before all `N_POINTS` are read); the NI warning `200010` that this raises is intentionally suppressed in that one place, since the early stop is deliberate.

3 The PFL-102 and SCC serial control

The PFL-102 is the two-stage SQUID flux-locked loop (FLL); its buffered output has a range of $\pm 10\,\mathrm{V}$ (voltage gain 5040) and reaches the DAQ through the PCI-1000 and DAC-2568. Control is one-directional: the PC sends commands down the PCI-1000 RS-232 link; there is no readback.

3.1 Lock state, and what "railed" really means

The PFL has two relevant states per stage, **LOCK** and **TUNE**:

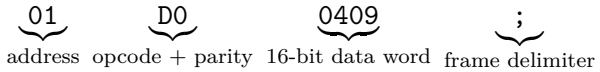
- In **TUNE** the feedback loop is *open* (used to set up the SQUID). The output then shows the SQUID's bounded, periodic voltage–flux characteristic — a small modulated signal, *not* a saturated rail. The noise can even be measured directly in this state.
- In **LOCK** the output tracks the applied flux. A **RESET** only has an effect in LOCK mode: it briefly opens the loop and discharges the integrator so tracking restarts as close to zero as possible.

Consequently, an unlocked output is small and bounded; a *railed* ($\sim \pm 10\,\mathrm{V}$) output is a *locked* loop that has run out of feedback range — the integrator driven past the $\pm 10\,\mathrm{V}$ buffer limit by large applied flux or drift, an "off-scale" condition cleared by a RESET. The automation's reset-and-recover logic therefore assumes the channel is locked; if a channel dropped to TUNE, a RESET would do nothing.

3.2 The SCC serial protocol

Commands are sent as ASCII text over a write-only link at 9600 baud, 8 data bits, no parity, 1 stop bit (8N1).

Command frame. Two hex address digits, two hex opcode digits, four hex data digits, ending in a semicolon — format `"%02X%02X%04X;"`:



Parity. Odd parity over the set bits of (address + opcode + data), placed in the most-significant bit (bit 7) of the opcode byte. The low seven bits of that byte are the opcode itself.

Addressing. A PFL on channel N has SCC address N (channel 1 = `0x01`); the PCI-1000 itself is `0xFF`.

3.3 The reset command

A reset is two frames: *assert* (data = register with bit 0 set) then *release* (data = register, returning to locked operation). With channel 1, opcode `0x50`, and the standard register `0x408`, the bytes on the wire are:

	frame on the wire	meaning of bit 0
assert	`01D00409;`	reset = on
release	`01500408;`	reset = off (locked operation)

(The parity bit makes the opcode byte `0xD0` for the assert frame and `0x50` for the release.) The reset toggles only bit 0 and preserves the rest of the register — it does not change any DAC values.

The feedback register (`0x50` data word). The standard configuration is `0x408`. Decoded against the PFL-102 register bit-fields (below), the only set bits are bit 10 and bit 3, giving: reset off; feedback resistor 100 kΩ; integrator 1.5 nF; test input off; and input-SQUID *Lock* with array *Tune* — i.e. SQUID-locked, high sensitivity, test off.

Table 2: PFL-102 feedback-loop register (opcode `0x50`).

Bits	Field	Encoding
`0`	Reset	`0` = off, `1` = reset
`2,1`	Feedback resistor	`00` = 100 kΩ, `01` = 1 kΩ, `10` = 10 kΩ
`9,8`	Integrator	`00` = 1.5 nF, `01` = 15 nF, `10` = 150 nF
`7-4`	Test input	`0000` = off, `0001` = SQUID bias, `0010` = array bias, `0100` = SQUID flux, `1000` = array flux
`3` and `11,10`	Tune/Lock	SQUID Lock + array Tune: `11,10`=`01`, bit 3=`1`. SQUID Tune + array Lock: `01`, `0`. Both Tune: `10`, `0`.

Caution. Confirm `cfg.register` (set via `register_override`, else the standard `0x408`) and `channel` match the actual front-panel lock state before each run. The release frame writes the *whole* register back, so a mismatch silently reprograms the PFL instead of merely resetting it. If you are array-locked, on a different sensitivity, or a different channel, set `register_override` / `channel` accordingly.

3.4 Detecting a failed reset

Two independent checks guard the reset:

1. **Verify read.** Immediately after the reset, a short read confirms the output cleared (|mean| < `baseline_v`). If it never clears after several retries, the run is a reset failure and the sweep stops.
2. **Baseline-at-start.** The opening chunks of each acquisition are watched; if the output is off-zero there, the reset "did not hold" and the run is flagged as a bad baseline.

The first asks *"did the reset clear it now?"*; the second asks *"is it still clear when the run begins?"* A slipped lock that briefly looked clear during the verify read is caught by the second check.

4 Structure of the run

Within a single scan interval, `run_cycle` repeats the loop below until it has collected `n_trials` clean traces or used its acquisition budget (`max_attempts`). The setup before the loop — resuming from any traces already on disk, then the initial reset — is omitted here.

```
repeat until  (N_TRIALS clean traces)  OR  (MAX_ATTEMPTS acquisitions used):

    acquire one consecutive run    # N_POINTS @ fs; prints start + expected-done
         |                         # jump-checked every CHUNK pts; aborts on a jump
         v
     < outcome? >
         |
         +-- clean .......... save  DAQ_..._{i}.txt (+ _temp.csv) ; log CLEAN ; n_clean++
         |                    -> loop again       (no reset -- the lock is good)
         |
         +-- jump/surge/rail  save  ..._{i}_{JUMP|SURGE|RAIL}.txt ; log ; n_attempts++
         |                    -> reset_and_verify
         |                          cleared?   yes -> loop again
         |                                     no  -> STOP the whole sweep
         |
         +-- bad_baseline ... save  ..._{i}_BADBASE.txt   (reset did not hold)
                              -> STOP the whole sweep

when the loop ends:   n_clean >= N_TRIALS ?   yes -> DONE
                                              no  -> BUDGET_EXHAUSTED  (next interval)
```

Here `i` is the on-disk order index: it increments on *every* acquisition, so a re-run resumes without overwriting. A reset fires only after a failed acquisition, never between clean traces; a reset that will not clear — or a bad baseline at the start (the reset did not hold) — stops the whole sweep.

5 Operating procedure

Caution. The finite-acquisition path is new on this hardware. Run the one-time bring-up (Section 5.1) as a supervised dry run before trusting it for science data; do not "Run All" blind.

5.1 One-time bring-up

Do these once, or whenever the rig or PC changes. Each is a known unknown that can silently kill a run.

- **Package import.** `import AutoSQUID as sq` resolves from the notebook (it adds the package parent to `sys.path`); the bench PC has `nidaqmx`, `pyserial`, and `RanLabPythonRepo` installed.
- **Serial port.** `cfg.port` defaults to `COM3`; confirm it with the bench-test notebook. PCS102DA can stay open on a different port (e.g. COM2) to hold the SQUID lock — just not on `COM3`.
- **Temperature backend.** Confirm `sq.read_temp(cfg)` returns the MXC value in kelvin and that channel 6 is the mixing chamber; compare to the Bluefors display.

- **DAQ smoke test.** Confirm a finite `samps_per_chan = 10_000_000` task is accepted and returns real (non-zero) data. Run one short acquisition first (e.g. `n_points = 100_000`), then one full 10 M run; confirm the full sample count comes back.
- **Save root.** Confirm the data root folder for your `user` exists (`cfg.outdir`).

5.2 Before every run (per temperature)

1. **Lock the FLL** on the channel you will use: SQUID-locked (stage 1), high sensitivity (100 kΩ / 1.5 nF). This is the per-cooldown manual calibration — do it well; the run depends on a good lock.
2. **Confirm `cfg.register` / `channel` match that lock** (Section 3.3).
3. **Baseline near zero.** With the loop locked, the output should sit within ±0.1 V of zero so the post-reset verify passes and the trace is centered. Do not nudge the flux off zero.
4. **Set and settle the temperature.** The label and the start temperature are read at the moment the run begins.
5. **Temperature logging is running**, so the reader returns a fresh value.
6. **Keep PCS102DA off this script's COM port.** It can stay open to hold the SQUID lock as long as it is on a different port (e.g. COM2) than this script (`COM3`); two programs on the same port collide.

5.3 Run order

Run the workflow in this order. Do not "Run All" on a new setup.

1. **Confirm wiring and lock state.** Use the wiring diagram (Figure 1). The SQUID/PFL output should reach the PCIe-6320 analog input, and the SCC serial path should control the PFL channel. Lock the input SQUID first — either in PCS102DA or with `sq.s_lock(cfg)`.
2. **Center the locked output.** Open `auto_s_tune.ipynb`, set `port`, `channel`, `daq_ai`, and `vrange`, then run:

   ```
   sq.s_lock(cfg)
   res = sq.auto_s_tune(cfg, start_sflux=..., target_V=0.0)
   ```

   Continue only if the returned `status` is `converged`, or if `dac_limited` is acceptable for the run. `no_response` means the live mean did not move with the S-flux steps — check the lock, `daq_ai`, and wiring (Section 6).
3. **Prepare the acquisition config.** Copy `example_measurement_cycle.ipynb` (or use the lab-local measurement notebook). Set at minimum `data_root`, `user`, `date`, `temp_reader`, `scan_interval_s`, `n_points`, `n_trials`, and `port`. If `temp_label="auto"`, `cfg.temp_reader` must be set before resolving the label.
4. **Detect or pin the DAQ input.** Run the checks cell:

   ```
   dev, cfg.daq_ai = sq.detect_ai_channel(cfg)
   ```

If auto-detect cannot identify the live channel, fix the lock/wiring or set `cfg.force_ai`.

5. **Resolve the temperature label.** Run `sq.resolve_temp_label(cfg)`. This reads the MXC temperature when `temp_label="auto"` and writes the rounded label into `cfg.temp_label`. For a resumed run, use the same literal label as the existing files if needed.

6. **Start acquisition, interval by interval.**

```
for tau in cfg.scan_intervals:
    if sq.run_cycle(cfg, tau) == "reset_fail":
        break
```

`run_cycle()` creates the output directory, verifies/reset-checks the lock, acquires traces, writes the PCS102 files, logs temperature, and resumes from existing files by date-agnostic trace core.

7. **Watch the console and logs.** During the run, monitor the outcome lines (`CLEAN`, `JUMP`, `SURGE`, `RAIL`, `BAD_BASELINE`) and the expected-done timestamps. Afterward, inspect `experiment_log.txt` and `action_log.txt`.

5.4 During the run

Each interval keeps acquiring until it has `n_trials` clean traces or uses up `max_attempts` acquisitions. Every acquisition is numbered in order; a clean one is saved without a suffix, a failed one with a `_JUMP`, `_RAIL`, or `_SURGE` suffix. Messages to expect:

- **Cleared** — after a reset, the 1000-point verify read shows the locked baseline back near zero ($|\text{mean}| <$ `baseline_v`, i.e. $< 0.1\,\text{V}$): the latched integrator discharged and the flux lock recovered, so acquisition begins. `reset_and_verify` fires up to five resets and prints `CLEARED` on the first that passes.
- **Clean** — the full `n_points` run completed *and* passed the post-hoc `is_surge_spec` check (no rail, opening mean in range, no $> 6\sigma$ baseline excursion); saved without a suffix, the clean count advances, and no reset is fired — the lock is already good.
- **Jump** — caught *live*, mid-run: a chunk's mean slipped from the established baseline by more than `jump_v`. The partial up to that point is saved with `_JUMP`, the loop is reset, and the interval re-acquires (one `used` attempt). At $\pm 1\,\text{V}$ a slip that reaches the clip reads as a $\sim 1\,\text{V}$ jump, so it lands here rather than as a rail.
- **Rail** — caught *live*: a chunk's $|V|_{\max}$ exceeded `rail_v` — the output pinned at a true rail (off-scale / lost lock). Saved with `_RAIL`, reset, re-acquire. Dormant at $\pm 1\,\text{V}$ (the ADC clips below `rail_v`, so a clipped slip is a Jump); active only if the card returns to $\pm 10\,\text{V}$.
- **Surge** — the run finished all `n_points` (nothing caught live), but the post-hoc full-trace `is_surge_spec` check failed: a baseline excursion past 6σ of the opening baseline (or an opening mean already off-zero) — a slow slip the live check let through. Saved with `_SURGE`, reset, re-acquire.
- **Bad baseline / reset is not holding** — detected in the first chunk(s), *before* the baseline is set: the trace started off-zero ($|\text{mean}| >$ `baseline_v`) or railed, i.e. the reset did not actually hold although its short verify passed. Treated as systemic — the diagnostic is saved and logged, then the sweep stops immediately, no retry. Fix the reset and re-run (it resumes from disk).

- **Done / Budget exhausted** — the interval reached `n_trials` clean traces, or used all `max_attempts` real acquisitions with fewer (the lock kept slipping); either way it moves to the next interval (re-run to top up, or raise `max_attempts`).

Long quiet stretches between the 30 s temperature prints are normal — a 5000 s run is not frozen.

Stopping mid-run. Interrupting the kernel during an acquisition stops at the next chunk boundary — after the current `chunk`-point read returns, not instantly. The in-progress trace is *discarded, not written* (only acquisitions that finished this run are on disk), and no reset is fired.

How the run's counters relate. The header on each acquisition shows three quantities, each determined differently:

- **order index `i`** — the highest index already on disk for this trace's core (`interval_temp_npts`, *ignoring* the month-day prefix) plus one, then +1 on every acquisition. It is read from disk, so it *continues* across re-runs, stops, and days; it is never reset or reused, and it is what the ledger's `attempt` column and the trailing filename number record (a clean run after two failures is still `_3`).
- **`used` (out of `max_attempts`)** — real acquisitions consumed *this run* (clean or jump/surge/rail, but not resets and not a bad-baseline abort). A per-run counter that starts at zero each re-run; the loop stops at `max_attempts`.
- **`clean` (out of `n_trials`)** — clean traces so far, seeded from the count already on disk for the core and incremented on each clean trace; the interval is done at `n_trials`.

The loop continues while `clean` $<$ `n_trials` *and* `used` $<$ `max_attempts`, so `index` $\geq$ `clean`: `index` advances on every acquisition, while `used` and `clean` advance only on the qualifying ones.

A run prints something like the following — note the measurement start time and, for every attempt, that attempt's start and expected-done time (result lines trimmed for width):

```
temperature label for this run: 15mK
measurement START : 2026-05-31 14:00:00
  reset try 1: mean=+0.0021 V -> CLEARED

=== DAQ_100us_15mK_10Mpts · index 1 · 0/4 used · 0/2 clean · 10000000 pts @ 10000 Hz (~1000 s) ===
    start 14:00:01 · expected done ~14:16:41  (~1000 s)
  CLEAN · idx 1 · N=10000000 · 1001s · mean=+0.0120 · 1/2 clean

=== DAQ_100us_15mK_10Mpts · index 2 · 1/4 used · 1/2 clean · 10000000 pts @ 10000 Hz (~1000 s) ===
    start 14:16:43 · expected done ~14:33:23  (~1000 s)
  JUMP · idx 2 · N=5300000 · 530s · mean=+0.5100 · 1/2 clean
  reset try 1: mean=+0.0019 V -> CLEARED

=== DAQ_100us_15mK_10Mpts · index 3 · 2/4 used · 1/2 clean · 10000000 pts @ 10000 Hz (~1000 s) ===
    start 14:25:40 · expected done ~14:42:20  (~1000 s)
  BAD_BASELINE · idx 3 · N=300000 · 30s · mean=+0.9980 · 1/2 clean
  bad baseline -> reset is not holding (systemic); STOPPING.
```

Every line is also appended, one row per acquisition, to the run ledger `experiment_log.txt`. The timestamp is a full ISO date-time; an abbreviated sample (key columns; the full set is listed under *After the run* below):

```
timestamp            outcome       clean att  mean_V    filename
2026-05-31T14:16:42  CLEAN         1     1    0.012000  DAQ_100us_15mK_10Mpts_1.txt
2026-05-31T14:25:38  JUMP          1     2    0.510000  DAQ_100us_15mK_10Mpts_2_JUMP.txt
2026-05-31T14:26:13  BAD_BASELINE  1     2    0.998000  DAQ_100us_15mK_10Mpts_3_BADBASE.txt
```

The `BAD_BASELINE` row is the tell: its `N` (console) is tiny — only the opening chunks were read — and the mean sits near 1 V, so the reset did not hold. Its index continues the sequence (3, after 1 and 2) and the sweep then stops.

5.5 After the run

1. **Check the output folder** for the expected clean traces — now named with a month-day acquisition-date prefix, e.g. `DAQ_Jun01_4us_4K_10Mpts_1.txt` — each with its temperature CSV, plus any failed traces, the run ledger (`experiment_log.txt`), and the results-free event log (`action_log.txt`). You should have `n_trials` clean files per interval.
2. **Read the ledger** (`experiment_log.txt`): one row per acquisition, with the outcome, the running clean count, reset count, and per-trace start/end temperature; the `attempt` column is the on-disk order index, which continues across re-runs and stops. The companion `action_log.txt` records only the event stream — resets, the temperature read just before each measurement, each `MEASURE` attempt, and bad-baseline — never any trace data.
3. **Re-check integrity before downstream analysis.** Acquisition already screens each trace with `is_surge_spec` before labeling it CLEAN — rails, $> 6\sigma$ baseline jumps, and frozen/stuck flat segments (flagged when a chunk's std collapses below the live noise level) — but a standalone re-check before PSD/analysis is still good practice.
4. **Record the cooldown context.** Traces are saved in raw volts with no flux-calibration factor or sample ID; note which cooldown, sample, and calibration factor they belong to so the right factor is applied later.
5. **To collect more traces or finish an interrupted interval**, re-run the active cell with the same settings: it counts the clean files already on disk, matching by the date-agnostic core (interval, temperature, point-count) so traces from *any* day in the folder all count and the index continues across days, and acquires only the shortfall, numbering from the next free index, so nothing is overwritten. The date prefix is ignored for matching, but the temperature label is part of the core, so if you used `"auto"` labeling set `temp_label` to the literal label the run produced before resuming.

5.6 Troubleshooting

Symptom	**Likely cause → fix**
Reset is not working / `RESET_FAIL` in the ledger; sweep stopped	A reset would not hold (this is by design systemic). Check the port (`COM3`; make sure PCS102DA is not on it) and that `cfg.register`/`channel` match the lock; confirm the lock and cable. Fix, then re-run (resumes from disk).
DAQ error, or the trace is all zeros	Finite 10 M buffer rejected, wrong range, or a copy-vs-view read issue — run the DAQ smoke test; switch to continuous mode if needed.

Symptom	Likely cause → fix
No live channel in the checks cell	SQUID not locked, wrong device, or output not on this card — confirm the lock and wiring; set `force_ai` if you know the channel.
Run crashes at the temperature-label line	The temperature reader returned nothing (logger not running / wrong channel).
Fewer clean files than `n_trials`	The interval hit the attempt budget (lock kept slipping). Re-run to top up, or raise `max_attempts`.
A bad-baseline trace and the sweep stopped	The reset is not holding (same causes as a reset failure). Fix the reset, then re-run.
Run looks frozen	Normal between 30 s temperature prints; a 500 μs run is $\sim$83 min.

5.7 Known limitations

- At the ± 1 V range the ADC clips at ~ 1 V, so any excursion past ~ 1 V reads as a ~ 1 V step and is reported as a `JUMP`. `rail_v` stays at 9.5 (unreachable here, so `RAIL` is dormant at ± 1 V) and remains valid if the card returns to ± 10 V; anything above ~ 1 V is discarded.
- Failed traces, the temperature CSVs, the ledger, and the action log live in the same folder as clean traces; when batch-loading, exclude the failed traces and the logs. Clean traces are exactly those with a bare integer index before `.txt` (no outcome word); matching ignores the month-day date prefix (keyed on the interval/temperature/point-count core).
- One temperature label is baked into all intervals of a run; on a long multi-interval sweep the label can drift from the true value — the per-trace start/end temperatures in the ledger are authoritative.

6 Auto-S-tune: centering the locked output

Before acquiring, the locked output is centered near zero so the baseline-near-zero precondition (Section 5) is met. Manually this is done by nudging the SQUID-flux control until the continuously-acquired trace looks centered around 0 mV. `auto_s_tune` (in `auto_s_tune.ipynb`) automates exactly that.

6.1 What it does

With the input SQUID *already locked*, it steps the SQUID-flux DAC (the `set_squid_flux` current, SCC `0x60` sub-opcode `0x0`) and, after each step, takes a short *finite* "live-view" read (`live_mean`, default 1 s) of the output mean. It moves toward the target by a **secant** update,

$$\Delta x \;=\; \frac{V_{\text{target}} - \bar{V}}{\text{slope}}, \qquad \text{slope} = \frac{\bar{V}_n - \bar{V}_{n-1}}{x_n - x_{n-1}} \text{ (V per } \mu\text{A)},$$

so the sign and gain of the response — unknown and per-cooldown — are measured, not assumed. Each step is clamped to `max_step_uA` so the lock cannot slip a fluxon. This mirrors OpenSQUID's `zeroStage1Output`, and uses the same finite DC reads (not a continuous stream). There is *no reset between steps*: while locked, the flux shifts the baseline directly — matching both the manual workflow and OpenSQUID.

6.2 Stop conditions

It stops when the live mean is within `tol_V` of target (default 3 mV — the "looks centered" criterion; on a 1 s read the mean's standard error is ∼0.02 mV, so this band is stable, not chasing noise), confirmed by one further read to reject slow drift. As a fallback it also stops when the predicted step falls below one DAC LSB (∼0.024 μA) — it cannot move finer (status `dac_limited`). It returns a status dictionary: `status` ∈ {`converged`, `dac_limited`, `no_response`, `max_iter`}, plus `flux_sflux`, `mean_V`, `std_V`, and `n_iter`.

6.3 Procedure

1. Close PCS102DA on the control port (or keep it on a different port).
2. In `auto_s_tune.ipynb` §0, build `cfg` (`port`, `channel`, `daq_ai`, `vrange`).
3. §1: `sq.s_lock(cfg)` to lock the input SQUID (or lock in PCS102DA), and make sure the trace is already *roughly* centered — within the ±1 V range.
4. Run `res = sq.auto_s_tune(cfg, start_sflux=..., target_V=0.0)`. Check `res["status"]`: `converged` means the output now sits within `tol_V` of zero at the returned `flux_sflux` (`dac_limited` means as close as one DAC LSB allows, still outside `tol_V`).

Caution. If the live mean does not respond to the S-flux steps (flat slope), `auto_s_tune` returns `no_response` rather than a false centering — typically the SQUID is not locked, `daq_ai` is wrong, or the S-flux DAC is not reaching the SQUID. Run `auto_s_tune` right after a manual lock that is already roughly centered, and it fine-tunes to zero.

Note. The initial reset is *on* by default (`reset_first=True`, OpenSQUID's single pre-reset); set `reset_first=False` to center from the live locked state without a pre-reset.